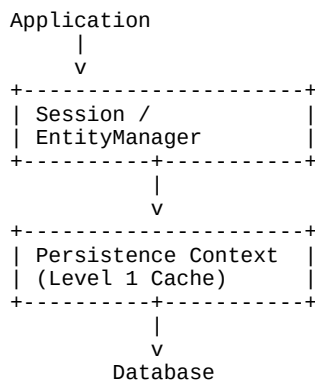


Chapter 9 - Persistence Context & Dirty Checking

1. Persistence Context

A Persistence Context is the collection of managed entity instances associated with a Hibernate Session (or JPA EntityManager). It tracks entity state changes and contains the Level 1 Cache.

2. Internal Architecture



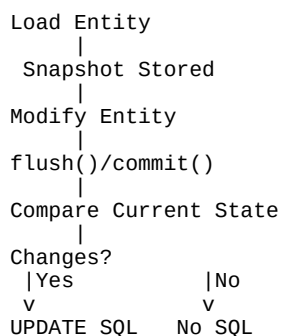
3. Managed Entities

When an entity is loaded using `get()`, `load()`, or `getReference()` (after initialization), it becomes a managed entity inside the Persistence Context. Hibernate automatically tracks modifications.

4. Dirty Checking

Dirty Checking is Hibernate's mechanism for detecting changes made to managed entities. Hibernate compares the current state with the snapshot captured when the entity became managed.

5. Dirty Checking Flow



6. Snapshot

Hibernate stores an internal snapshot of each managed entity. During flush, the current values are compared with this snapshot. If differences exist, an UPDATE statement is generated.

7. Example

```
Employee emp=session.get(Employee.class,1);  
emp.setName("Siva");  
transaction.commit();  
  
// Hibernate automatically executes UPDATE
```

8. SQL Generated

```
SELECT * FROM EMPLOYEE WHERE ID=1;  
  
UPDATE EMPLOYEE  
SET NAME='Siva'  
WHERE ID=1;
```

Action	Persistence Context Result
get()	Entity becomes managed
persist()	Entity becomes managed
clear()	All entities detached
evict(entity)	One entity detached
close()	Persistence Context destroyed

9. Best Practices

- Keep the Persistence Context short-lived.
- Avoid very large Sessions because many managed entities increase memory usage.
- Rely on Dirty Checking instead of manually issuing UPDATE statements for managed entities.

10. Interview Questions

1. What is a Persistence Context? 2. Where is the Persistence Context stored? 3. What is Dirty Checking? 4. What is an entity snapshot? 5. When does Hibernate perform Dirty Checking? 6. Why is no update() call required for managed entities?